МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра информационной безопасности

ОТЧЕТ

по лабораторной работе №2

по дисциплине «Архитектура система»

Тема: Реализация спроектированного приложения

Студентка гр. 3362 _____ Беседина А.К.

Преподаватель _____ Троценко В.В.

Санкт-Петербург

2026

Цель работы.

Реализовать приложение выбранной предметной области по спроектированной ранее чистой архитектуре. В рамках работы необходимо разделить систему на слои domain, use cases, ports, adapters и infrastructure, интерфейсы и зависимости между компонентами, а также показать работоспособность приложения через выбранный HTTP-интерфейс.

Задачи на лабораторную работу.

1. Введение. Краткое описание продукта и воркфлоу (из Практики 1).
2. Схема архитектуры.
3. Составить таблицу Маппинг «схема → код».
4. Таблица слоёв. Сводная таблица с количественными характеристиками.
5. Проверка endpoint’ов.
6. Сделать выводы по работе.

ВЫПОЛНЕНИЕ РАБОТЫ

1. Введение. Краткое описание продукта и воркфлоу (из Практики 1).

Проект представляет собой сервис бронирования отелей. Пользователь открывает веб-страницу, указывает город, даты проживания и число гостей, после чего система находит свободные номера. Затем пользователь создаёт временную бронь (hold), вводит данные гостя и подтверждает бронь. После подтверждения карточку брони можно открыть по ID, а при необходимости - отменить.

По сравнению с идеей из Практики 1 реализация была смещена от REST + JSON к fullstack HTML over HTTP: сервер отдаёт не только полные страницы, но и HTML-фрагменты, которые HTMX подставляет в нужный блок интерфейса. Внутренние слои при этом остались теми же: domain, ports, use cases, adapters, infrastructure. Изменения затронули главным образом входные адаптеры и формат HTTP-контрактов.

Для акцента на архитектурной прослеживаемости выбрано in-memory хранилище.

2. Схема архитектуры.

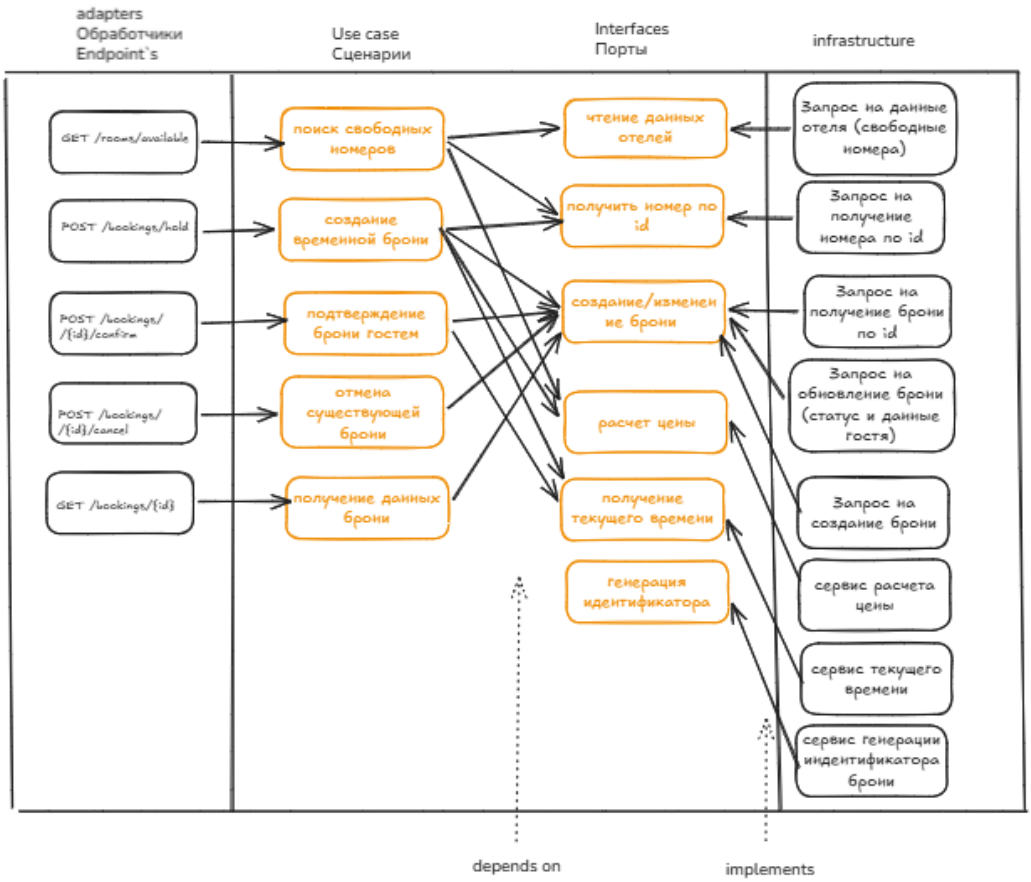

Рисунок 1 – схема архитектуры

Структура проекта:

```
app/
├── domain/
├── ports/
├── usecases/
├── adapters/
├── repositories/in_memory
├── infrastructure/
└── main.go
```

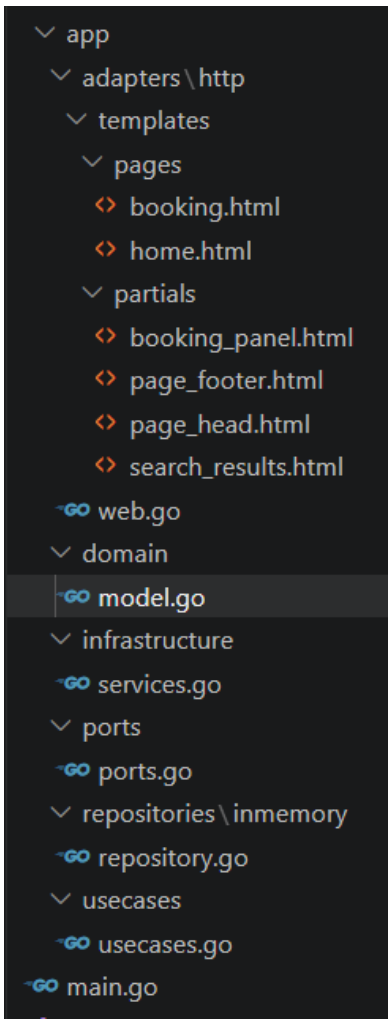

Рисунок 2 – схема проекта

3. Маппинг «схема → код».

Слой	Компонент на схеме	Файл / пакет	Зависит от (imports)
Domain	Room / Guest / Booking / AppError / Moment	app/domain/model.go	-
Ports	AvailabilityReader, AvailabilityChecker, RoomReader, BookingRepository, PriceCalculator, Clock, IDGenerator	app/ports/ports.go	app/domain
Use Cases	FindAvailableRoomsUseCase, HoldBookingUseCase, ConfirmBookingUseCase, CancelBookingUseCase, GetBookingUseCase	app/usecases/usecases.go	app/domain, app/ports, time
Adapters	GET /rooms/available POST /bookings/hold GET /bookings/{id} POST/bookings/{id}/confirm POST/bookings/{id}/cancel	app/adapters/http/web.go	app/domain, app/ports, app/usecases, net/http, html/template
Infrastructure	InMemoryRepository	app/repositories/inmemory/repository.go	app/domain, sync, time
Infrastructure	SystemClock PriceCalculator IDGenerator SeedRooms	app/infrastructure/services.go	app/domain, time, crypto/rand, encoding/hex
Infrastructure	Composition root / ручной DI	main.go	app/adapters/http, app/usecases, app/infrastructure,

			app/repositories/inmemory, net/http, time

4. Таблица слоёв.

Слой	**Ответственность**	**LOC**
Domain	Сущности, value objects, бизнес-правила, доменные ошибки, тип Moment	219
Use Cases	Сценарии поиска, hold, confirm, cancel и чтения брони	207
Ports	Интерфейсы репозиториев и внешних сервисов	36
Adapters	HTTP-обработчики, HTML templates, HTMX-compatible responses	450
Infrastructure	In-memory repository, сервисы времени/цены/ID, composition root main.go	227

5. Проверка endpoint'ов.

Поиск свободных номеров по городу, датам и числу гостей, отправляем GET-запрос **GET /rooms/available** на адрес hhtp://localhost:8080 с определёнными параметрами.

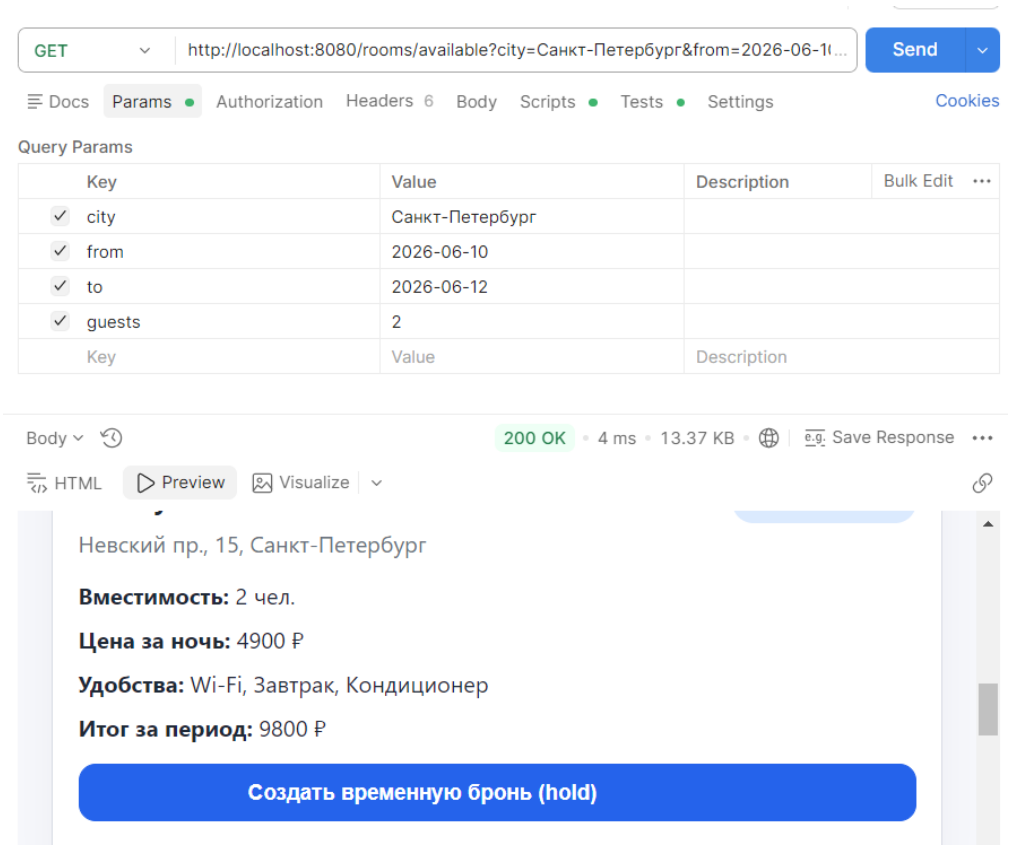

Рисунок 3 – запрос на получение отелей

Вывод: успешный, варианты номеров.

POST /bookings/hold. После того как пользователь выбрал отель, следует создание временной брони (hold) для свободного номера.

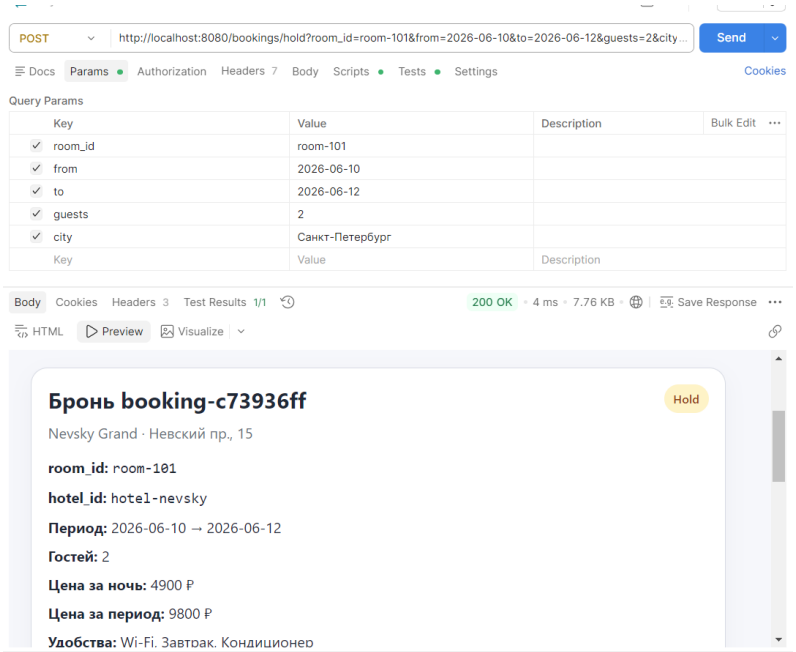

Рисунок 4 – запрос на бронирование отеля

POST /bookings/{id}/confirm. Подтверждение hold с корректными данными гостя.

POST http://localhost:8080/bookings/booking-c7499c78/confirm Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cool

none form-data x-www-form-urlencoded raw binary GraphQL

Key	Value	Description	Bulk Edit
first_name	Fyyf		
second_name	Беседина		
phone	8921091237		
email	aaAAAA@htmx.com		
Key	Value	Description	

Body Cookies Headers 3 Test Results 1/1 200 OK 4 ms 4.58 KB Save Response

HTML Preview Visualize

Бронь booking-c7499c78 Confirmed

Nevsky Grand · Невский пр., 15

room_id: room-101 **hotel_id:** hotel-nevsky

Период: 2026-06-10 → 2026-06-12 **Гостей:** 2

Цена за ночь: 4900 ₽ **Цена за период:** 9800 ₽

Удобства: Wi-Fi, Завтрак, Кондиционер **Статус:** Confirmed

Рисунок 5 – успешное подтверждение брони

Ошибочный сценарий: отправляем POST-запрос с пустым именем гостя, в ответ получаем ошибку, так как имя гостя обязательно вписать.

POST http://localhost:8080/bookings/booking-a1c3f28c/confirm

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL

Key	Value	Description
first_name		
second_name	Беседина	
phone	8921091237	
email	aaAAAA@htmx.com	
Key	Value	Description

Body Cookies Headers 3 Test Results 0/1 400 Bad Request 4 ms 5.29 KB Save

HTML Preview Debug with AI

Бронь booking-a1c3f28c

Ошибка валидации

имя гостя обязательно

Рисунок 6 – ошибка при вводе данных гостя

Ошибочный сценарий: прошло много времени для заполнения данных гостя и временная бронь уже истекла.

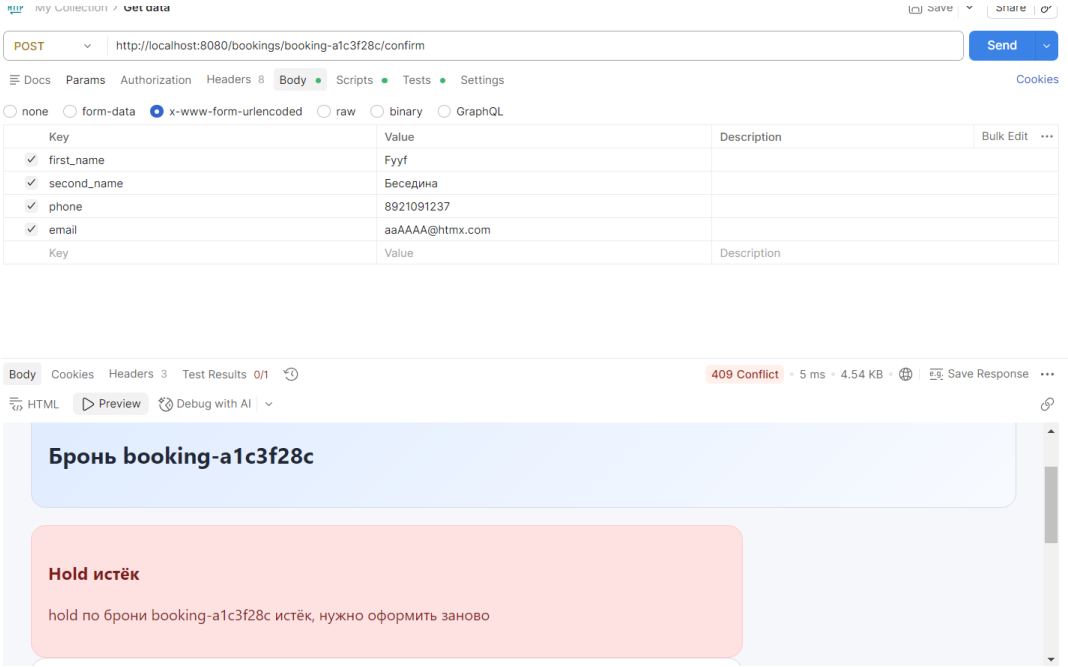

Рисунок 7 – подтверждение истёкшего hold

POST /bookings/{id}/cancel.

Успешный сценарий: отмена существующей брони.

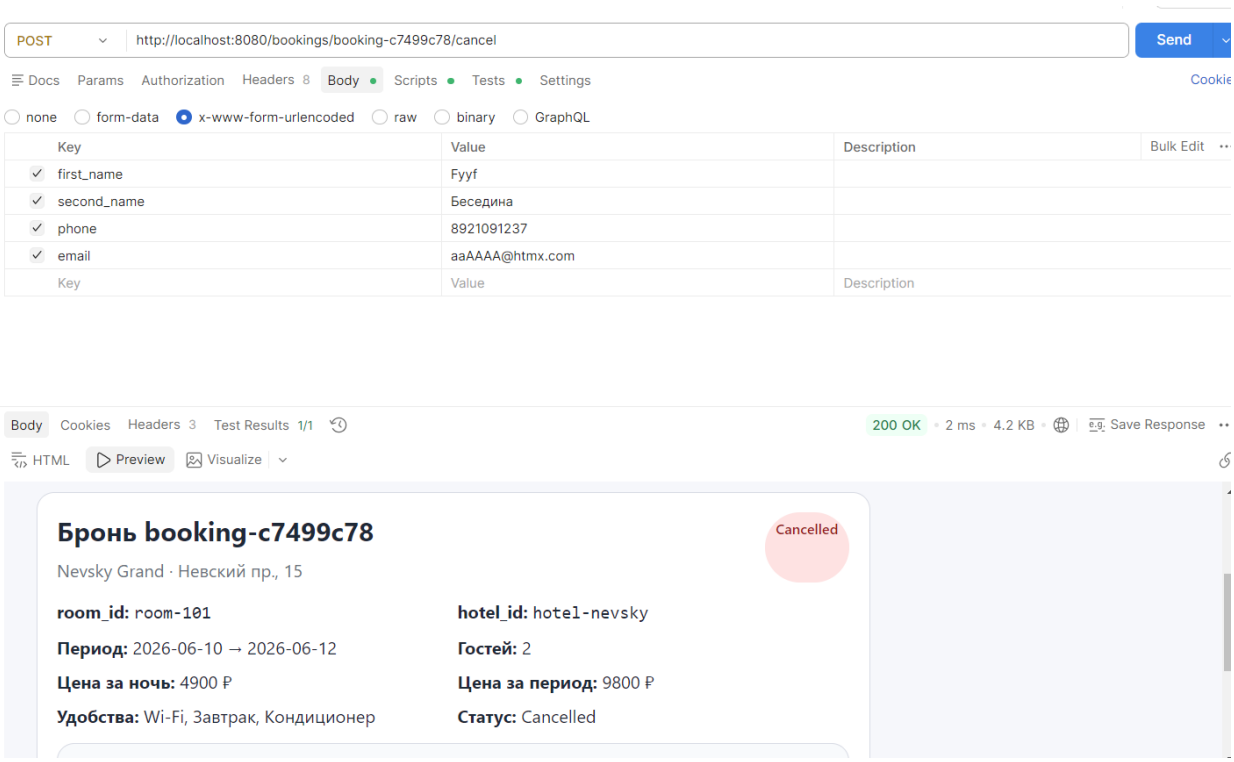

Рисунок 8 – отмена существующей брони

Ошибочный сценарий: повторная отмена уже отменённой брони.

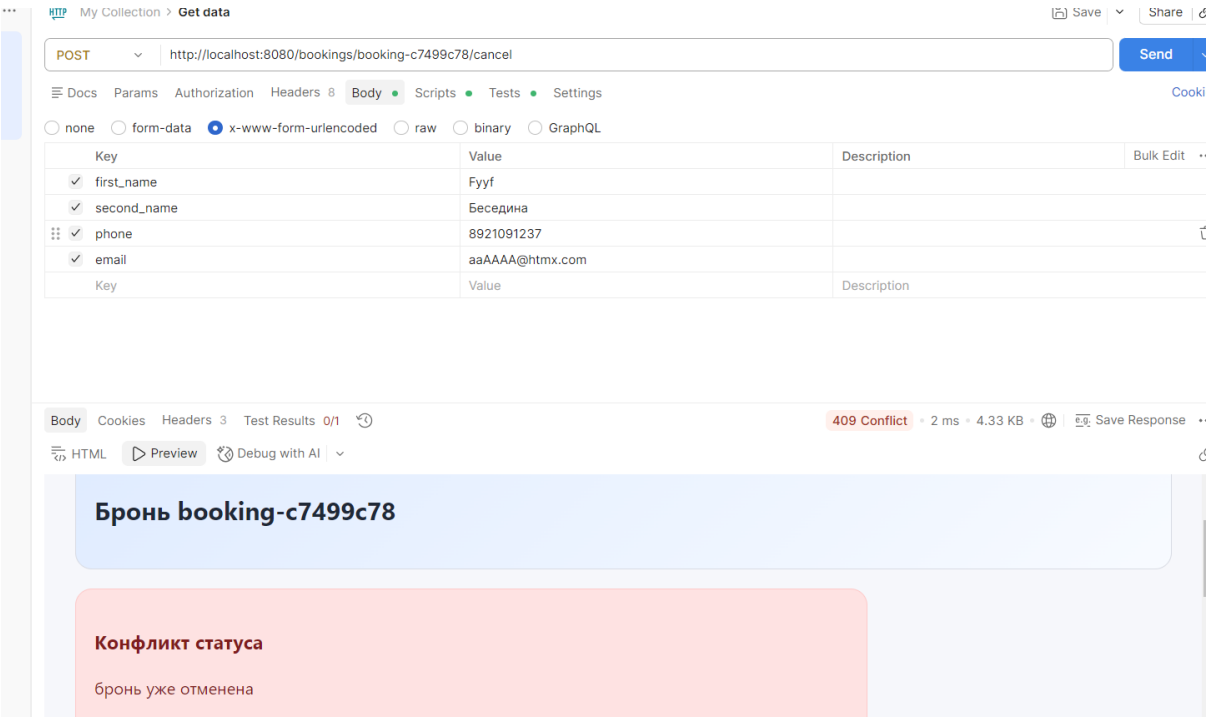

Рисунок 9 – повторная отмена брони

GET /bookings/{id}. Получение карточки брони по ID, возможность отменить бронь.

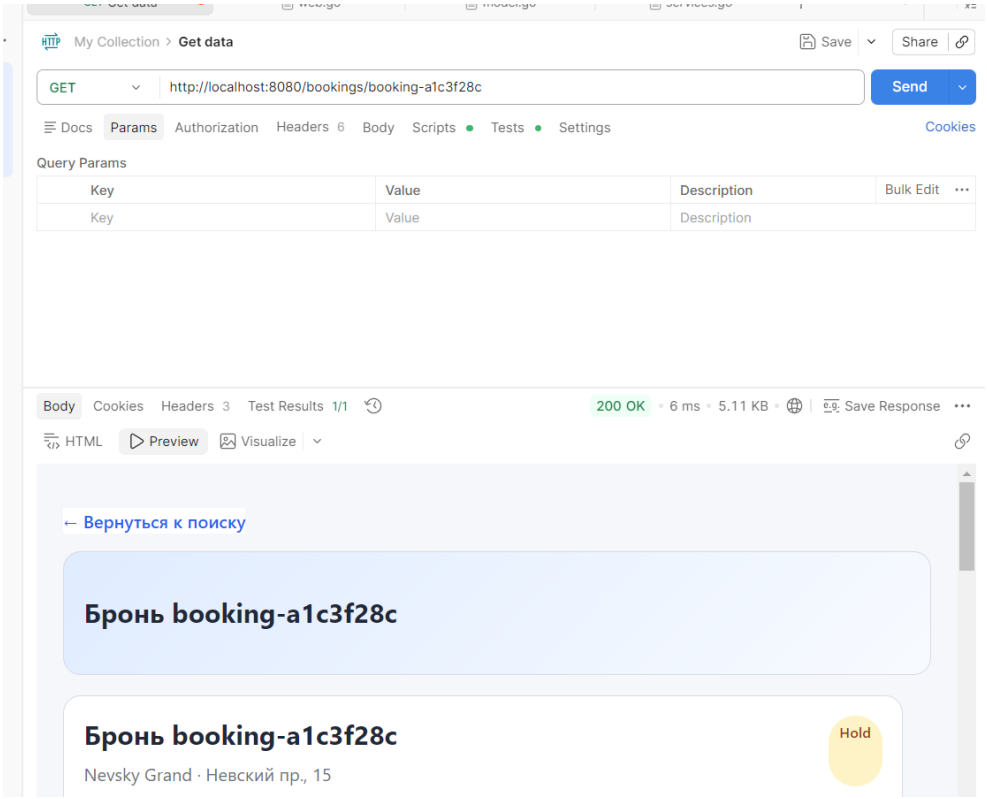

Рисунок 10 – информация о брони по ID

Выводы.

В ходе работы реализовано приложение сервиса бронирования отелей в соответствии с архитектурой, спроектированной в первой лабораторной работе. Система построена с разделением на слои domain, use cases, ports, adapters и infrastructure, что позволило отделить бизнес-логику от деталей хранения данных. Для взаимодействия выбран fullstack HTML over HTTP, а работоспособность приложения подтверждена через Postman. Хранение данных выполнено в памяти, без использования внешней базы данных. Также были определены ключевые интерфейсы и направления зависимостей между компонентами, а сборка приложения выполнена через ручное внедрение зависимостей в main.go.

ПРИЛОЖЕНИЕ А – Команды запуска

В терминале папки проекта:

go run .

В браузере:

http://localhost:8080

ПРИЛОЖЕНИЕ Б – ВИЗУАЛЬНЫЙ ИНТЕРФЕЙС

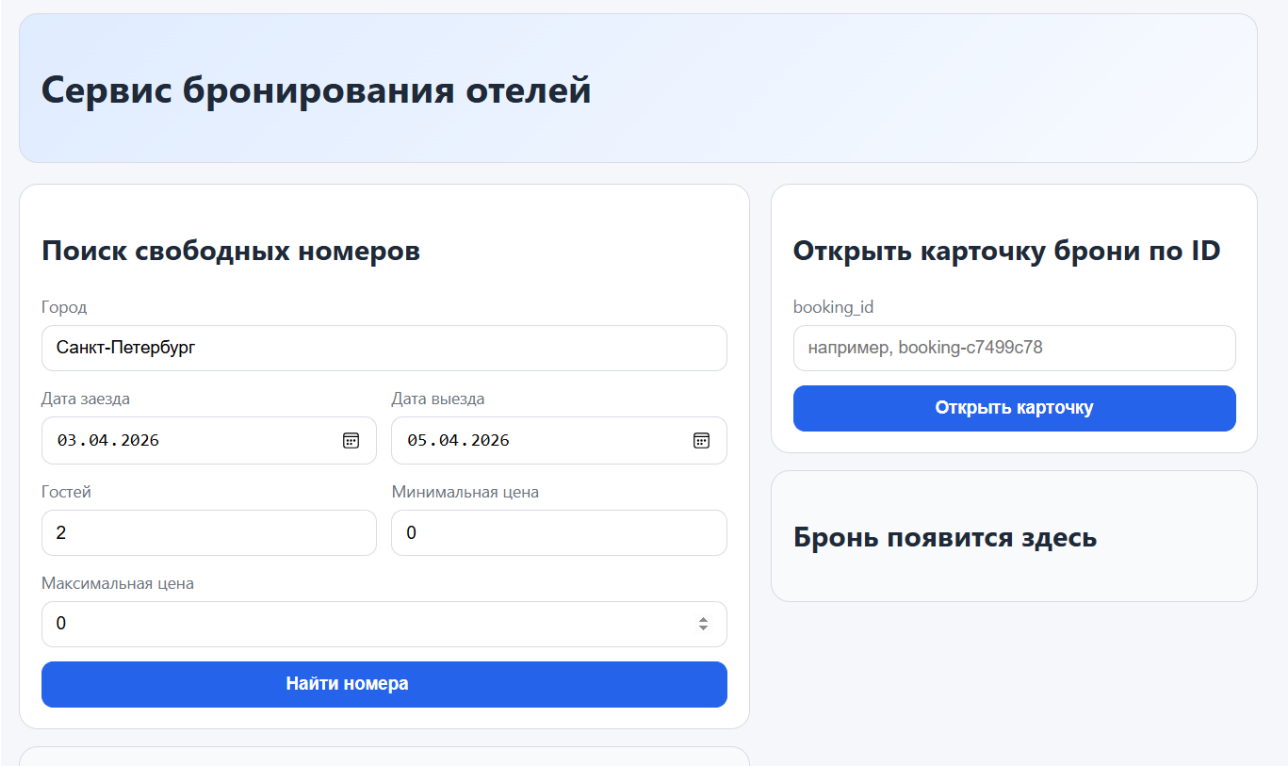

Рисунок 1 – изначальная страница

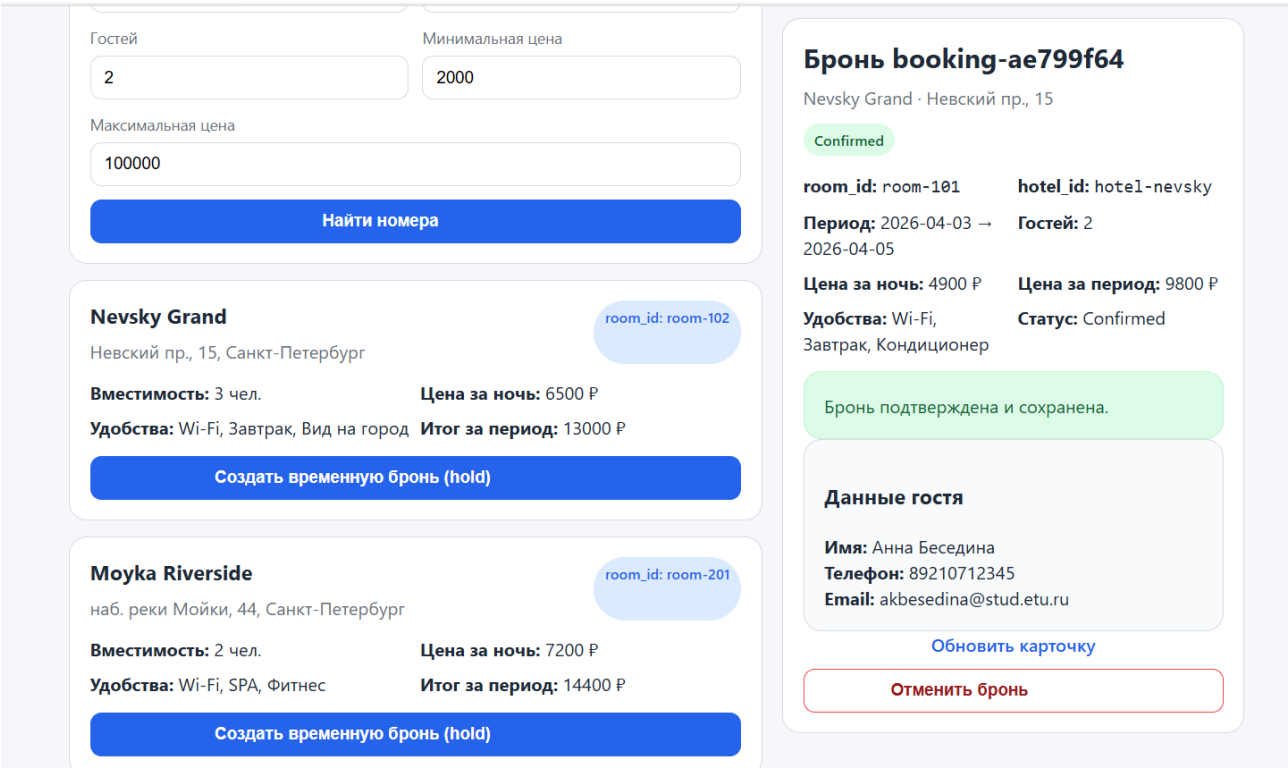

Рисунок 2 – выполненная бронь